As you can see, you insert the new element on either side of a target or as a child (first or last) of the target. Let's use `AdjacentPosition::BeforeBegin`…

```
$target = $dom->querySelector('main p:nth-child(2)');
var_dump($target->textContent);
$target->insertAdjacentElement(Dom\AdjacentPosition::BeforeBegin, $img);
print $dom->saveHTML();
```

```
Output:
string(8) "Para Two"
...
    <main class="main-content">
      <p class="large">Para One</p>
      <img src="./jess.jpg" alt="A picture of Jess the cat">
      <p class="small  ">Para Two</p>
      <p class="small">Para Three</p>
    </main>
...
```

… and as we can see the `img` element we created with `Dom\Document::createElement()` has been inserted before the `p` target we specified with our `main p:nth-child(2)` selector.

Time for Coffee

This concludes our deep dive into the DOM. You should now have a good feel for what HTML really is – a serialized XML string that represents a tree data structure of element nodes that follows a specification (the HTML schema). We covered the types of elements you'll encounter and where they should live in the schema (e.g., `meta` elements only live in `head`), and that elements have special metadata called attributes – particularly, the important `class` and `id` attributes.

We saw how to create a DOM structure with `Dom\HTMLDocument` (or `DOMDocument`) and how to traverse it with XPath queries or the new CSS-driven `Dom\Document::querySelector*()` methods, which included a crash course in XPath and CSS syntax. We also saw how to manipulate the DOM by moving elements around, changing their attributes, and removing and adding new elements in the DOM. That's a fair bit to digest, so please feel free to relax with a beverage of your choice before we move onto the next section, where we'll build HTML from scratch.